

红石双核计算机硬件与指令集规范说明书

零、程序存储器架构

1. 硬件架构与功能概述

- 双核独立对称存储**：主核（Master Core）与从核（Slave Core）各自配备一套物理独立的 3 Bank ROM 阵列，两颗核心的指令空间互不干扰，整机总程序容量高达 **192 条指令**（主核 96 条 + 从核 96 条）。修改硬件的情况下整机总程序容量理论上限是 **320 条指令**
- 单核 3 Bank 拓扑划分**（主从两核完全相同）：
 - Bank 0（基准程序空间）**：寻址范围为 5 位地址 00000 到 11111（0 到 31），用于存放核心主程序。
 - 拓展 Bank（程序拓展空间）**：寻址范围为 5 位地址 00000 到 11111（32 到 63），与 Bank 0 结合构成 64 条指令的连贯单任务逻辑空间。
 - Bank 1（独立副程序/协程空间）**：寻址范围为 5 位地址 00000 到 11111（0 到 31），拥有独占的 32 条指令容量，用于存放子程序、中断服务例程（ISR）或并发协程任务。
- 寻址总线**：每个 Bank 具备独立的 5 位寻址总线，程序计数器（PC）按模 32 循环执行。

2. 硬件使用规范与注意事项

- 环形 PC 隔离**：在默认顺序执行状态下，PC 执行到地址 31（11111）后，下一周期将按模 32 自动回绕至当前空间的地址 0（00000），**绝不会自动溢出至其他bank**。
- 空间穿梭约束**：跨越 32 条指令边界进入拓展空间必须使用 EXT 指令配合跳转；进入 Bank 1 则必须通过 YIELD 主动触发上下文切换程序或者被外部中断抢占。
- 编码规范说明**：本指令集所有未使用的缺省位、操作码保留位**一律严格填 0，严禁写入任意值或通配符**，以防硬件译码器产生未定义的逻辑漂移。

一、寄存器

1. 硬件架构与功能概述

- 累加器 A（ACC / Register A）**：系统的主通用数据寄存器，位宽为 3 位，数值范围为 0 ~ 7。作为 ALU 运算的主操作数和默认结果写回目标，所有算术运算均按模 8 自动回绕溢出。两核各自配备独立的寄存器 A。
- ALU B 寄存器（Register B）**：ALU 的副操作数寄存器，物理映射于内存单元 101。

- **主核**：该单元只可写入不可读出。
- **从核**：该单元物理上实现读写分离，写入作为 B 寄存器，读取则来自单向硬件邮箱。
- **基址寄存器 (Base Register)**：物理映射于内存单元 100，位宽为 3 位。该单元本身为通用数据 RAM，专门兼作 LDR / SDR 间接寻址的硬件基地址。如果不需要存储基址，直接当正常寄存器用即可。主从两核各自独立。
- **程序计数器 (PC)**：位宽为 5 位，各核心独立维护。

2. 相关指令列表及详细说明

- **LDDI (立即数加载到 A)**
 - **机器代码**：00001XXX
 - **详细说明**：将指令中的 3 位立即数 xxx (0~7) 直接加载到当前核心的寄存器 A 中。
- **ADDI (立即数加法到 A)**
 - **机器代码**：00010XXX
 - **详细说明**：将 3 位立即数 xxx 与寄存器 A 当前的值相加，结果写回寄存器 A。运算遵循 3 位模 8 回绕规则 (例如：当前 A 为 6 时执行 ADDI 3，计算 $6 + 3 = 9 \equiv 1 \pmod{8}$ ，结果 A 变为 1)。
- **LDDIR (立即数直接加载到 B 寄存器)**
 - **机器代码**：01001XXX
 - **详细说明**：直接将 3 位立即数 xxx 写入当前核心的内存单元 101 (ALU B 寄存器)，为后续的 ALU 运算准备操作数 B。该指令完全不改变寄存器 A 的值。
- **SW 101 / 写 B 寄存器**
 - **机器代码**：00100101
 - **详细说明**：SW 指令写地址 101 的助记符。将寄存器 A 的当前值存入当前核心的 B 寄存器 (内存 101)。
- **SW 100 / 写基址寄存器**
 - **机器代码**：00100100
 - **详细说明**：将寄存器 A 的当前值写入内存单元 100，设置后续 LDR / SDR 间接寻址所使用的基地址。

3. 硬件使用规范与注意事项

- **B 寄存器预置要求**：ALU 属于两操作数运算单元，在执行加法、减法、比较、赋值、按位与和右移之前，必须先通过 LDDIR 或 SW 101 设置好 B 寄存器的值，否则 B 寄存器的内容处于未定状态，导致运算逻辑错误。
- **模 8 溢出无进位标志**：系统无独立的溢出标志位 (Carry Flag)，数值达到 8 会自动归零，软件若需实现多字节大数运算，需在应用层算法中通过条件比较判断进位。

二、 内存（数据存储器）

1. 硬件架构与功能概述

- **物理阵列划分：**主核与从核各自拥有独立的 8 单元物理 RAM 阵列，地址宽度为 3 位，编码范围为 000 到 111，每个单元数据位宽为 3 位（数值 0~7）。
- **主核内存拓扑映射：**
 - 000：主核私有通用数据单元（仅主核可见）。
 - 001 ~ 100（4 个单元）：**跨核一致性共享数据窗口**，兼作 DMA 缓冲区。
 - 101：ALU B 寄存器（只写单元）。
 - 110：南桥模块选择地址端口（只写单元）。
 - 111：南桥模块数据发送端口（只写单元）。
- **从核内存拓扑映射：**
 - 000：从核私有通用数据单元。
 - 001 ~ 100（4 个单元）：**跨核一致性共享数据窗口**。
 - 101：写入为从核 ALU B 寄存器；读取为单向硬件邮箱输出端口。
 - 110 ~ 111：**释放为从核独占的私有通用数据单元**（从核可自由读写，无 MMIO 占用）。

2. 相关指令列表及详细说明

- **LW（读取内存到 A）**
 - **机器代码：** 00011XXX
 - **详细说明：** 读取内存地址 xxx 的数据并载入寄存器 A。
- **SW（写入寄存器 A 到内存）**
 - **机器代码：** 00100XXX
 - **详细说明：** 将寄存器 A 的当前值写入内存地址 xxx。
- **LDR（基于基址寄存器的间接读取）**
 - **机器代码：** 00110XXX
 - **详细说明：** 硬件自动读取内存单元 100 中保存的基地址，与指令中的 3 位立即数偏移 xxx 相加并模 8，得到目标绝对地址，将该目标地址中的数据读入寄存器 A。
- **SDR（基于基址寄存器的间接写入）**
 - **机器代码：** 00111XXX
 - **详细说明：** 硬件将内存单元 100 的基地址与 3 位偏移 xxx 相加并模 8 得到目标绝对地址，将寄存器 A 的当前值写入该目标地址。

3. 硬件使用规范与注意事项

- **主核 LW 访问禁区：**在主核上，普通访存指令 LW 严禁访问 101、110、111 这三个单元。这三者在主核上连接了专用的外设/寄存器总线，违反时虽然不会停机，但会触发外设读取导致逻辑崩溃。

- **从核 110/111 访问合法性**：从核由于没有南桥电路，其 110 和 111 已转为通用私有 RAM，从核执行 LW 110、LW 111、SW 110、SW 111 完全合法且零 IPI 产生。
- **间接寻址边界防御**：使用 LDR 和 SDR 时，目标地址 $([100] + XXX) \pmod{8}$ 若在主核中落入 101、110、111，硬件不会拦截。程序员必须主动规划偏移量，确保寻址范围严格受控于合法数据区（如 000 ~ 100）。
- **数组访问标准范式**：访问以 000 为起始地址的数组第 3 个元素（下标 3），先执行 LDDI 000 → SW 100 设置基址，再执行 LDR 011 即可。

三、硬件栈

1. 硬件架构与功能概述

- **系统包含两套物理完全独立、互不干扰的硬件栈**：
 - **硬件调用栈 (Call Stack)**：深度最大为 **3 层**。专用于保存和恢复程序计数器 PC 与寄存器 A，由 CALL、RET 以及中断和上下文切换微码硬件自动管理。
 - **硬件数据栈 (Data Stack)**：专用于函数内部临时变量暂存、寄存器数据倒手交换 (Swap) 与局部表达式计算，由 PUSH 与 POP 指令显式操作。
- 两颗核心各自配备一套独立的调用栈与数据栈。

2. 相关指令列表及详细说明

- **CALL (子程序调用)**
 - **机器代码**：101XXXXX
 - **详细说明**：调用目标 5 位地址 xxxxx 处的子程序。硬件自动将当前程序计数器 PC（指向下一条指令）和累加器 A 的当前值压入硬件调用栈保存，随后将 PC 跳转到 xxxxx。
- **RET (子程序返回)**
 - **机器代码**：10000000（未使用的低 5 位严格填充为 00000）
 - **详细说明**：子程序返回。从硬件调用栈栈顶弹出并恢复此前保存的 PC 与寄存器 A 的值，使控制流精准返回到当初 CALL 调用点的下一条指令继续运行。
- **PUSH (压入数据栈)**
 - **机器代码**：01100000
 - **详细说明**：将寄存器 A 的当前 3 位数值压入硬件数据栈顶部，不改变寄存器 A 的值。
- **POP (弹出数据栈)**
 - **机器代码**：01100001
 - **详细说明**：从硬件数据栈顶部弹出一个 3 位数据并写入寄存器 A。

3. 硬件使用规范与注意事项

- **调用栈 3 层硬限制**：全局范围内任意时刻，CALL 的嵌套调用深度（含中断嵌套、子程序与上下文切换微码中占用的 CALL）**绝对不得超过 3 层**！超过 3 层会导致调用栈溢出，破坏最底层的返回地址与现场数据。
- **数据栈作用域平衡铁律**：数据栈在同一个函数或代码块内部必须严格保证 **PUSH 的次数与 POP 的次数完全相等**。若不平衡会导致数据栈溢出或在后续数据弹出时发生数据错位。
- **两栈隔离意识**：RET 永远恢复来自 CALL 的数据，POP 永远恢复来自 PUSH 的数据，切勿混淆两者的生命周期。

四、 算术逻辑单元 (ALU)

1. 硬件架构与功能概述

- 位宽为 3 位，针对寄存器 A（主操作数）与寄存器 B（副操作数，RAM 101）进行组合逻辑运算，结果写回寄存器 A。
- 运算类型由 3 位操作码 OPQ 决定，支持加法、减法、小于比较、赋值、按位与以及右移六大功能。

2. 相关指令列表及详细说明

指令统一前缀机器代码为 01000OPQ：

- **ALU SUB（减法运算，OPQ = 001）**
 - **机器代码**：01000001
 - **详细说明**：计算 $A = (A - B) \pmod{8}$ 。当减法产生借位时按模 8 回绕（例如： $A = 2, B = 3$ ，计算 $2 - 3 = -1 \equiv 7 \pmod{8}$ ）。
- **ALU CMP（小于比较运算，OPQ = 010）**
 - **机器代码**：01000010
 - **详细说明**：比较寄存器 A 与 B 的大小。若 $A < B$ 则结果写回 $A = 0$ ；若 $A \geq B$ 则结果写回 $A = 1$ 。
- **ALU ADD（加法运算，OPQ = 011）**
 - **机器代码**：01000011
 - **详细说明**：计算 $A = (A + B) \pmod{8}$ 。结果按模 8 自动溢出回绕。
- **ALU MOV（赋值操作，OPQ = 100）**
 - **机器代码**：01000100
 - **详细说明**：将 B 寄存器（RAM 101）中的数值原样复制到寄存器 A 中（ $A = B$ ）。
- **ALU AND（按位与运算，OPQ = 101）**
 - **机器代码**：01000101
 - **详细说明**：将寄存器 A 与寄存器 B 进行 3 位逐位逻辑与运算（ $A = A \& B$ ）。

- **ALU SHR (逻辑右移运算, OPQ = 110)**
 - **机器代码:** 01000110
 - **详细说明:** 将 B 寄存器中的数值向右逻辑平移 1 位, 最高位补 0, 结果存入寄存器 A, 即计算 $A = \lfloor B/2 \rfloor$ 。

3. 硬件使用规范与注意事项

- **操作码纠偏确认:** 硬件底层译码器真值表严格定义 001 为减法 (A-B), 011 为加法 (A+B), 切勿颠倒。
- **比较指令与 BEQ 的天然搭配:** ALU CMP 在条件成立 ($A < B$) 时专门输出数字 0, 这一特性可以直接配合 BEQ (为 0 跳转) 指令实现标准高级语言中的 if (A < B) goto target; 分支控制。

五、IO 设备

1. 硬件架构与功能概述

- **交互权限归属:** 所有的物理人机交互外设 (键盘输入) 与外部扩展模块 (南桥总线设备) **由主核独占管理与访问**, 从核在电路上不连接这些外设总线。
- **键盘 MMIO:** 采用内存映射 I/O, 通过专用读取总线映射到地址 110 与 111。
- **南桥 MMIO:** 提供 8 个外部模块的寻址接口 (地址 000 到 111), 主核通过配置内部内存单元 110 (模块地址) 与 111 (发送数据), 配合专用的 MMIO 指令激活时钟信号进行通讯。

2. 相关指令列表及详细说明

- **LW 110 (读取键盘输入标志位)**
 - **机器代码:** 00011110
 - **详细说明:** 主核专用。读取键盘输入就绪标志到寄存器 A。当有玩家按键按下时, 标志位为 1; 无按键输入时为 0。该指令不清除标志位。
- **LW 111 (读取键盘键值)**
 - **机器代码:** 00011111
 - **详细说明:** 主核专用。读取键盘数据缓冲区中的 3 位键值到寄存器 A。**硬件在读取发生的瞬间, 会自动执行读清机制, 将键盘就绪标志位强制复位为 0, 为下一次按键做好准备。**
- **MMIO (南桥模块激活)**
 - **机器代码:** 01111000
 - **详细说明:** 主核专用。硬件读取内存单元 110 的值作为目标模块地址, 读取单元 111 的值作为数据载荷, 向南桥目标模块发送一个时钟激活脉冲, 完成一次物理发送。
- **LW 101 (主核: 读取南桥模块数据)**
 - **机器代码:** 00011101

- **详细说明：**主核专用。硬件根据当前内存单元 110 中的南桥模块地址，读取对应南桥模块当前输出的 3 位数据到寄存器 A。

3. 南桥模块地址分配表与网络读清铁律

模块地址 (3-bit)	模块名称与功能说明	读写特性与标志位行为 (含自动清空与禁止二次读取规则)
000	有线网卡发送模块 (TX)	写入：将数据写入 111，地址设为 000，执行 MMIO 发送
001	有线网卡接收数据缓冲区 (RX Data)	读取：地址设为 001，执行 LW 101 读取。 ⚠️ 读清机制： 读取瞬间，硬件自动清空数据缓冲区， 并同时 将接收标志位 (010) 强制复位为 0！ ⚠️ 禁止二次读取： 数据已被清空， 绝对不能二次读取该地址！ 读取完毕后硬件立即就绪， 可以直接接收下一个网络数据包。
010	有线网卡接收标志位 (RX Flag)	读取：地址设为 010，执行 LW 101 读取（1 为有数据到达，0 为无包）。
011	USB 输入 1 / 预留控制端口	专用扩展控制端口。
100	USB 输入 2 兼 RSLINK 标志位	读取：地址设为 100，执行 LW 101 轮询互联接口是否有数据到达。
101	无线网卡发送模块 (TX)	写入：将数据写入 111，地址设为 101，执行 MMIO 发送。
110	无线网卡接收数据缓冲区 (RX Data)	读取：地址设为 110，执行 LW 101 读取。 ⚠️ 读清机制： 读取瞬间，硬件自动清空数据缓冲区， 并同时 将接收标志位 (111) 强制复位为 0！ ⚠️ 禁止二次读取： 数据已被清空， 绝对不能二次读取该地址！ 读取完毕后硬件立即就绪， 可以直接接收下一个无线数据包。
111	无线网卡接收标志位 (RX Flag)	读取：地址设为 111，执行 LW 101 读取（1 为有数据到达，0 为无包）。

4. 硬件使用规范与注意事项

- **网络收发标准操作流：**
 - i. 将 010（或 111）写入 RAM 110，执行 LW 101 读取网络标志位；
 - ii. 若 A 为 0，利用 BEQ 循环等待；
 - iii. 当 A 为 1 时，立即将 001（或 110）写入 RAM 110，执行 LW 101 一次性读出有效载荷；
 - iv. 读出后**必须立刻把数据存入内部寄存器或 RAM（如 SW 000）保存**，绝不能再次执行 LW 101 读该地址（否则读出全 0 垃圾数据）。
- **键盘轮询标准规范：**读取键盘的标准软件流为：执行 LW 110 → 执行 BEQ 轮询等待 → 标志位变 1 后执行 LW 111 取值并自动读清标志。

六、 中断系统、上下文切换与中断嵌套

1. 硬件架构与功能概述

- **专用微码存储空间：**独立于主核与从核的 3 Bank ROM，拥有独立的切换程序存储器，分为**主动 YIELD 组**与**硬件键盘中断组（可以通过修改硬件连线切换中断源）**，每组包含 8 条指令槽位（前 4 条为程序 A，后 4 条为程序 B）。
- **切换机制：**
 - **程序 A（Bank 0 → Bank 1）：**负责挂起 Bank 0，保存现场并切入 Bank 1。
 - **程序 B（Bank 1 → Bank 0）：**负责结束 Bank 1，恢复现场并返回 Bank 0。
- **双模式调度：**支持传统子程序/中断模型（CALL / RET）与无栈并发协程模型（CSW）。
- **从核非抢占特性：**从核完全不配备抢占式中断，算力流程绝不受外部随机外设打断。

2. 中断优先级与中断嵌套体系（Interrupt Nesting）

- **严格优先级拓扑：**

硬件抢占式中断（键盘中断 / 从核中断 IPI） > 软件主动让出（YIELD）

- **全空间抢占响应能力：**
 - **无论主核当前是在 Bank 0 主程序，还是在 Bank 1 副程序/协程中运行，一旦发生键盘中断或从核中断，硬件一律立即响应，强制打断当前执行流！**
- **中断嵌套与逐级返回机制：**
 - **Bank 0 运行期间发生中断：**单层中断。硬件压栈 Bank 0 的 PC 与 ACC → 进入键盘/从核 ISR → 执行完毕触发 FINISH 经 RET → 精准返回 Bank 0 现场。
 - **Bank 1 运行期间发生中断：形成中断嵌套！** 硬件将当前 Bank 1 正在执行处的断点 PC 与 ACC 再次压入硬件调用栈（占用第 2 格调用栈）→ 强行切入抢占中断服务程序 → 抢占服务程

序执行完毕后执行 FINISH 经 RET → **优先精准返回 Bank 1 被中断处继续执行副程序** → 待 Bank 1 全部执行完毕执行自身的 FINISH → 再次经 RET 返回 Bank 0 主程序。

- **全局调用栈硬指标**：发生中断嵌套时，由于每次抢占式中断均使用 CALL 占用 1 格调用栈，加上程序内部自身的函数调用，**总嵌套深度绝对不能超过 3 层！**

3. 相关指令列表及详细说明

- **YIELD（主动任务出让触发）**
 - **机器代码**：01110000
 - **详细说明**：执行后硬件触发独立的上下文切换程序 A，启动由 Bank 0 向 Bank 1 的切换。其优先级低于硬件中断。
- **FINISH（副程序/中断服务结束触发）**
 - **机器代码**：01101000
 - **详细说明**：**严格限制只能在 Bank 1 中执行**。执行后硬件触发对应的上下文切换程序 B，启动由 Bank 1 切回 Bank 0（或切回被嵌套中断的上一级断点）的返回流程。
- **CSW（专用协程/上下文任务切换指令）**
 - **机器代码**：01101100
 - **详细说明**：配置于上下文切换程序 A 与 B 的末尾，替代传统的 CALL 与 RET。
 - **硬件行为**：**完全不占用硬件调用栈深度**；硬件维护双 PC 记忆。首次切入 Bank 1 强制从地址 00000 启动，后续来回切换自动记忆各自离开时的断点 PC 并断点续跑。
 - **现场保存**：硬件仅记忆双方 PC，**不自动保存累加器 A**，需由程序员按需手动通过 PUSH / POP 暂存。
- **HALT（停机挂起指令）**
 - **机器代码**：00101000
 - **详细说明**：终止当前核心的指令执行，使 CPU 进入完全静默的休眠挂起状态。

4. 硬件使用规范与注意事项

- **键盘中断固定 CALL/RET 铁律**：键盘中断切换程序组在硬件电路上强制要求固定采用 CALL 与 RET 机制，电路上不支持 CSW 指令，以保证抢占现场能通过调用栈实现严格的嵌套与逐层回退。
- **微码执行原子性**：上下文切换程序（微码）执行期间，硬件自动屏蔽新的中断响应，切换过程本身不会被打断。

七、显示屏

1. 硬件架构与功能概述

- **5 联屏物理拓扑**：显示终端由 **5 块物理独立的显示屏单元**（编号为 0 到 4 号）横向排列组成，支持同时呈现 5 个独立的数字、符号或图元。
- **硬件光标环形寻址**：光标默认支持原位保持与自动递增，**按模 5 循环步进**（ $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$ ），连续执行 5 次步进操作即可自动回绕到开头。**硬件电路不支持光标退格**。支持单周期硬件指令直接将光标拉回第 0 格开头。
- **15 帧自定义显存 + 1 专用空帧**：
 - 系统支持 4 位帧索引编码（*DABC*，范围 0000 到 1111，共 16 种状态）。
 - **0000 到 1110（共 15 帧）**：由用户自定义字模内容的有效显示帧（数字 0-7 固定映射在第 0-7 帧）。
 - **1111（第 16 帧）**：**硬件保留专用空帧**，向当前屏幕块写入该帧可实现单格内容的擦除。
- **独占控制权**：仅主核拥有显示器总线控制权限，从核无物理连接。
- **三模式驱动架构**：
 - **快速单周期控制模式**：极速控制流，提供独立的清空全屏与光标归首指令，单周期即时生效，大幅节约指令空间。
 - **旧版 3 位直显兼容模式**：极速显示，支持直接输出数字 1 ~ 7，**光标原地不动**。
 - **新版 6 位拓展控制模式**：完整控制流，通过两步 4 条指令发送 6 位复合数据，硬件自动解耦为 **4 位帧数据（*DABC*）与 2 位控制操作码（*EF*）**，实现图元渲染、光标步进与局部擦除。
- **辅助自定义画板屏**：系统除上述 5 联屏外，还配备了一块独立的 4 联屏显示器。
 - **4 帧图元选择**：该屏支持输出用户在屏幕背面通过红石火把自定义的 **4 帧独立图元**（编号为 00 到 11）。
 - **硬件寻址映射**：显示的目标帧编号直接由主核内存单元 **111 的低 2 位（[1:0]）** 决定。
 - **步进与锁死机制**：每次执行专用指令显示选中的一帧且光标自动后移 1 格，执行 4 次铺满屏幕后硬件锁定禁止继续显示，光标不循环回绕。
 - **外部手动清空**：该屏幕需由用户在外部手动拉动拉杆清空复位，**计算机没有清空或复位画板屏的软件指令**，与 5 联主屏物理独立、互不冲突。

2. 相关指令列表及详细说明

A. 5 联主显示屏单周期极速控制指令

- **CLEAR 5-SCREEN（主屏快速全屏清空）**
 - **机器代码**：01101010
 - **详细说明**：主核专用。单周期全字节精准译码指令。执行后瞬间清空 5 块主显示屏的全部内容，**光标原地保持不动**。

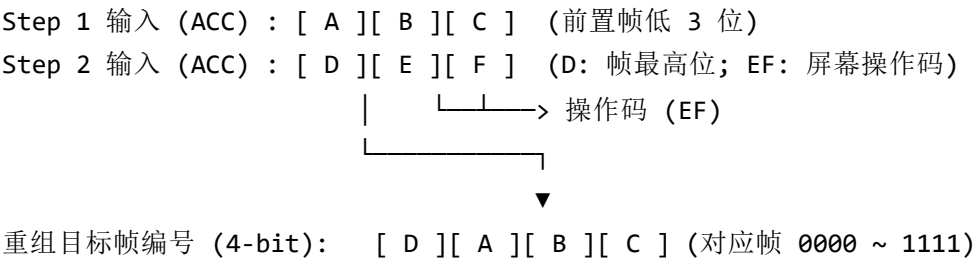
- HOME CURSOR (主屏快速光标归首)
 - 机器代码：01110010
 - 详细说明：主核专用。单周期全字节精准译码指令。执行后瞬间将 5 联主屏的光标强制拉回第 0 块屏幕开头，屏幕当前显示内容完全保留。

B. 5 联主显示屏数据与拓展显示指令

显示屏复合控制指令编码格式统一为 01010JKL：

- DISPLAY 3-bit (单次 3 位直显兼容指令)
 - 机器代码：01010100 ($J = 1, K = 0, L = 0$)
 - 详细说明：将寄存器 A 当前的 3 位数值直接送入当前光标指向的屏幕译码器并立即点亮。
 - 限制与特性：
 - 仅支持显示数字 1 ~ 7 (耗时仅 2 条指令：LDDI + 01010100)。
 - 执行后光标原地不动 (+0)。
 - 严禁用于显示数字 0 (由于电平特性限制，数字 0 无法被该指令触发，必须使用 6 位拓展指令)。
- DISPLAY 6-bit Step 1 (6 位拓展控制前半段暂存)
 - 机器代码：01010001 ($J = 0, K = 0, L = 1$)
 - 详细说明：硬件检测到 $J = 0, L = 1$ ，将当前寄存器 A 的 3 位数据作为前置信号 (标记为 ABC) 锁存到显卡输入暂存器中，屏幕此时不动作。
- DISPLAY 6-bit Step 2 (6 位拓展控制后半段触发)
 - 机器代码：01010111 ($J = 1, K = 1, L = 1$)
 - 详细说明：硬件检测到 $J = 1, K = 1, L = 1$ ，提取当前寄存器 A 的 3 位数据 (标记为 DEF)，与此前锁存的 ABC 重新线网交织，重组为 4 位帧内容 $DABC$ 与 2 位操作码 EF ，并在本周周期瞬间执行对应硬件动作。

复合 6 位数据重组映射关系



6 位模式操作码（EF）功能真值表

操作码 (EF)	完整第二步载荷 (DEF)	硬件动作名称	屏幕显示行为	光标状态流向
01	D01	绘帧并步进	在当前屏渲染帧 DABC 内容	向后步进 1 格（模 5 循环）
10	D10	绘帧并原位保持	在当前屏渲染帧 DABC 内容	原地不动
11	011 (D = 0)	全屏清空 (兼容)	5 块屏幕内容瞬间全清	原地保持 (建议改用极速指令 01101010)
11	111 (D = 1)	光标归首 (兼容)	5 块屏幕内容完全保留	复位至第 0 格 (建议改用极速指令 01110010)

C. 4 联自定义画板屏专用指令

- **DISPLAY CANVAS（自定义画板屏单帧步进输出）**
 - **机器代码：** 00101100
 - **详细说明：** 主核专用。硬件自动读取当前主核内存单元 111 的最低 2 位（[1:0]）作为图元帧索引（范围 00 ~ 11，共 4 帧），并在画板屏当前光标所在的格子上渲染该帧内容，随后光标自动向右移动一格。
 - **限制：** 连续执行 4 次后填满 4 块画板屏并自动硬件锁定终止响应，后续再次执行硬件不再显示，光标绝不循环回绕。

3. 硬件使用规范与注意事项

- **极速全屏复位新范式：**

由于新增了单周期控制指令，重置 5 联主显示屏（全屏变黑且光标回到最左侧开头）已从旧版的 8 条指令大幅精简至仅需 2 条指令：

01101010

； 1. 瞬间清空 5 联主屏全部内容（光标原地保持）

01110010

； 2. 瞬间将光标强行拉回第 0 格开头

LDDI 001 ; 准备帧索引 (例如加载第 1 帧, 二进制 001, 低两位为 01)
SW 111 ; 写入 RAM[111], 选定画板屏目标字模
00101100 ; DISPLAY CANVAS: 渲染第 1 帧到当前画板屏单元, 光标自动 +1

- **画板屏饱和锁死与外部手动清空特性:**

画板屏的光标为单向非循环计数器, 满 4 次后光标锁死于末端, 用户若需再次使用画板屏, 必须由操作员前往机身在外部手动拉下拉杆清空并复位硬件。

- **6 位控制四指令标准流水线 (主屏绘制) :**

在 5 联主屏上显示自定义图元或数字 0 时, 仍需按照严格的 4 指令流水线执行:

LDDI ABC ; 1. 准备 ABC 参数 (低 3 位字模)
01010001 ; 2. DISPLAY Step 1 (硬件锁存 ABC)
LDDI DEF ; 3. 准备 DEF 参数 (D 为最高位字模, EF 为操作码)
01010111 ; 4. DISPLAY Step 2 (触发执行显示与步进控制)

- **数字 0 渲染硬性规范:**

数字 0 映射于第 0 帧 ($DABC = 0000$)。严禁用 3 位直显指令 01010100 显示 0 (电平特性限制无法点亮)。在当前主屏幕显示数字 0 时, 必须使用 6 位模式: 第一步必须设 $ABC = 000$, 第二步 D 位必须设为 0:

- 显示 0 并移动光标到下一格: 第二步设 $DEF = 001$ ($D = 0, EF = 01$) ;
- 显示 0 且光标停留在原地: 第二步设 $DEF = 010$ ($D = 0, EF = 10$) 。

- **单格擦除范式:**

若仅需擦除主屏当前所在的一格而不影响其他 4 格, 需调用保留空帧 $DABC = 1111$ (即 $ABC = 111, D = 1$) :

- 单格擦除并移到下一格: $DEF = 101$ ($D = 1, EF = 01$) ;
- 单格擦除并停在当前格: $DEF = 110$ ($D = 1, EF = 10$) 。

- **两套屏幕设备完全独立互不冲突:**

5 联主终端与 4 联自定义画板屏在硬件总线与控制器上物理隔离。对 5 联主屏进行的任何显示、步进、清屏 (01101010) 或归首 (01110010) 操作均不会干扰画板屏; 反之, 执行 00101100 也完全不影响主屏的内容与光标。

八、DMA 设备

1. 硬件架构与功能概述

- **批量突发传输机制:** DMA 控制器直接接管物理总线, 绕过 CPU 算术单元, 在 RAM 单元 001 至 100 (固定 4 个存储单元) 与外部存储介质 (硬盘扇区、RSLINK 高速互联接口) 之间实现整块数据的成批高速搬运。

- 主核与从核均配备独立的 DMA 传输指令通道。

2. 相关指令列表及详细说明

指令编码格式统一为 01011XXB：

- **XX 目标扇区选择：**
 - 01：硬盘扇区 1 (Sector 1)
 - 10：硬盘扇区 2 (Sector 2)
 - 00：高速互联接口 RSLINK
- **B 传输方向标志位：**
 - B = 0 (**写入内存 / 读盘**)：从外部目标介质读取 4 个数据并**写入**到内存 001~100。
 - B = 1 (**读取内存 / 写盘**)：从内存 001~100 批量**读取** 4 个数据并写入/发送到目标介质。
- **严格机器代码定义 (无通配符)：**
 - **硬盘 01 扇区读取**：01011010 (后必须紧跟 00000000)
 - **硬盘 01 扇区写入**：01011011 (后必须紧跟 00000000)
 - **硬盘 10 扇区读取**：01011100 (后必须紧跟 00000000)
 - **硬盘 10 扇区写入**：01011101 (后必须紧跟 00000000)
 - **RSLINK 接收**：01011000 (后必须紧跟 00000000)
 - **RSLINK 发送**：01011001 (后必须紧跟 00000000)
- **NOP (硬件延迟槽)**
 - **机器代码**：00000000
 - **详细说明**：空操作。任何 DMA 指令后必须紧跟一条 NOP 作为硬件电路稳定的延迟槽。

3. 硬件使用规范与注意事项

- **非一致性 DMA 核心规则 (Non-Coherent DMA)：**
 - **底层硬件原理**：DMA 控制器直接对物理 RAM 进行批量数据灌入，完全绕过了 CPU 核心内部的常规 SW 写译码器，**因此 DMA 覆写内存绝不会使数据变脏，对应的硬件脏位 (Dirty Bit) 保持原样不变！**
 - **多核协同陷阱与解法**：若主核通过 DMA 从硬盘向共享窗口 001~100 写入了输入参数并希望从核感知，主核必须在 DMA 完成后**主动对目标单元执行一次轻触读写操作 (Touch)**，例如执行 LW 001 再立刻 SW 001，以此强行触发 CPU 写译码器将脏位置 1，否则从核后续读取时因脏位仍为 0 将直接读取本地过期数据。
- **基址寄存器保护提醒**：RAM 100 既是间接寻址的基址寄存器，又处于 DMA 的 001~100 覆盖范围内。执行 DMA 读盘会无条件覆写单元 100 的基地址，程序员必须在 DMA 操作后重新确认或恢复基地址。

九、地址拓展机构

1. 硬件架构与功能概述

- **分页拓展原理**：为了打破 5 位寻址总线最多只能寻址 32 条指令的物理上限，硬件在主核与从核中各自内置了一个 **T 触发器 (Toggle 翻转器)**。
- **空间整合**：通过翻转第 6 位地址线状态，将 Bank 0 (0~31) 与拓展 Bank (32~63) 打通为一个拥有 **64 条指令的连贯单程序空间**。
- 主核与从核内部的 EXT 机构完全独立，两核均能独立运行 64 条指令的单任务。

2. 相关指令列表及详细说明

- **EXT (地址拓展状态翻转前缀)**
 - **机器代码**：01110100
 - **详细说明**：将当前核心的地址拓展硬件触发器状态翻转 (0 变 1, 1 变 0)。当此指令执行后，紧随其后的下一条控制流跳转指令 (JMP、BEQ 或 CALL) 将被硬件导向另一个 32 条指令空间的对应 5 位地址中。

3. 硬件使用规范与注意事项

- **双指令配对法则**：EXT 本身不引发控制流跳转，它是一个修饰前缀。要想实现跨 Bank 远跳转，必须采用 **EXT + 跳转指令** 的双指令组合 (总共占用 2 个指令槽位)：
 - EXT 紧跟 JMP XXXXX：跨空间绝对远跳转 (Far JMP)。
 - EXT 紧跟 BEQ XXXXX：跨空间条件远跳转 (Far BEQ)。
 - EXT 紧跟 CALL XXXXX：跨空间子程序远调用 (Far CALL)。
- **对称双向穿梭**：从 Bank 0 跳入拓展 Bank (0~31 → 32~63)，以及从拓展 Bank 跳回 Bank 0 (32~63 → 0~31)，**全部使用相同的 EXT + 跳转 序列**。
- **零调用栈开销**：使用 EXT + JMP 或 EXT + BEQ 进行跨空间跳转完全不消耗硬件调用栈，珍贵的 3 层调用栈可全部留给复杂业务逻辑。

十、从核与双核协同体系

1. 硬件架构与功能概述

- **非对称主从定位**：
 - **主核 (Master Core)**：负责系统前台调度、UI 渲染、键盘交互与南桥外设控制，掌握系统最高调度权。

- **从核 (Slave Core)**：作为独立的**纯算力协处理器**，被物理剥离了键盘、显示器和南桥交互总线，拥有专属的私有 L1 缓存空间 (000、110、111)，专职高速执行数学计算与复杂算法。
- **共享与隔离拓扑**：
 - 001 ~ 100 (4 个单元)：带硬件脏位监测的双端口一致性共享窗口。
 - 000、110、111：从核私有存储，绝不触发 IPI，不可被主核直接观测。
- **双向通信非对称模型**：
 - **主核 → 从核**：参数下发走**硬件透明自动同步** (或单向硬件邮箱)。
 - **从核 → 主核**：结果交付走**显式 RSLINK DMA 传输**。

2. 相关指令列表及详细说明

- **WRMBX (主核：写入单向硬件邮箱)**
 - **机器代码**：01100100
 - **详细说明**：主核专用。硬件将累加器 A (ACC) 当前的 3 位数值直接锁存写入主向从单向硬件邮箱中。**该过程单周期零延迟、不占用 RAM 资源、绝对不触发 IPI**，适合寄存器级极速下发任务编号与控制参数。
- **LW 101 (从核：读取单向硬件邮箱)**
 - **机器代码**：00011101
 - **详细说明**：从核专用。从核执行此指令，单周期直接读出主向从硬件邮箱中的数据到累加器 A。
- **CPUTOG (主核：从核运行状态/电源翻转控制)**
 - **机器代码**：01111100
 - **详细说明**：主核专用。向从核发射启停控制脉冲 (Toggle 逻辑)：
 - 若从核处于关机/休眠状态，执行后立即**开机启动**；
 - 若从核处于开机/运行状态，执行后立即**强制定身关机**并进入休眠。
 - **关键硬件行为**：下达开机指令时，硬件会同时强制将从核的程序计数器 PC 复位归零！从核开机后永远默认从从核自身独立的 Bank 0、地址 00000 干净启动，绝不续跑历史断点。
- **从核交差发送指令**：DMA 01011001 配合 NOP 00000000，将从核 001~100 结果灌入 RSLINK。
- **主核提取结果指令**：DMA 01011000 配合 NOP 00000000，从 RSLINK 读取结果到主核 001~100。

3. 硬件协同运行规范与注意事项

- **单向硬件透明一致性机制 (Master → Slave)**：
 - 主核使用 SW 修改了 001~100 中任意单元，硬件自动将该单元标记为 Dirty；
 - 从核在自身代码中执行 LW 或 LDR 读取该单元时，硬件自动检测到脏位，**瞬间挂起从核时钟 (Stall)**，并向主核发出 IPI 核间中断；
 - 底层硬件自动完成数据同步更新并清零脏位，随后**自动解冻从核并重新执行刚才被挂起的读指令**；

- **对程序员完全透明**：从核编写者无需关心任何中断细节，直接写常规读指令即可保证读到最新数据。
- **显式交付铁律 (Slave → Master)**：
 - 自动脏位同步**严格仅限主向从单向**！从核向共享区写入结果，主核不会自动拉取；
 - 从核计算完成后，**必须显式执行 DMA 01011001 + NOP 00000000 把结果推入 RSLINK 接口**；
 - 主核在完成自己的任务后，通过将南桥地址设为 100，执行 LW 101 主动非阻塞轮询 RSLINK 标志位（数据到达时硬件不会打断主核）；
 - 查到标志位为 1 后，主核显式执行 DMA 01011000 + NOP 00000000 提取结果，并在完成后执行 01111100（CPUTOG）关停从核。

附录：全系统完整机器指令编码速查总表

归属硬件模块	指令助记符	8位机器代码	适用核心	核心功能与硬件动作摘要
基础数据传输	LDDI	00001XXX	双核通用	$ACC \leftarrow XXX$ (3位立即数加载)
	LW	00011XXX	双核通用	$ACC \leftarrow RAM[XXX]$ (主核限 000~100, 从核含 110/111)
	SW	00100XXX	双核通用	$RAM[XXX] \leftarrow ACC$
	LDDIR	01001XXX	双核通用	$Reg_B(RAM[101]) \leftarrow XXX$ (不改变 ACC)
算术逻辑单元	ALU SUB	01000001	双核通用	$ACC \leftarrow (ACC - B)$ (mod 8) (减法)
	ALU CMP	01000010	双核通用	若 $ACC < B$ 则 $ACC \leftarrow 0$, 否则 $ACC \leftarrow 1$
	ALU ADD	01000011	双核通用	$ACC \leftarrow (ACC + B)$ (mod 8) (加法)
	ALU MOV	01000100	双核通用	$ACC \leftarrow B$ (赋值)
	ALU AND	01000101	双核通用	$ACC \leftarrow ACC \& B$ (按位与)
	ALU SHR	01000110	双核通用	$ACC \leftarrow \lfloor B/2 \rfloor$ (逻辑右移 1 位)

归属硬件模块	指令助记符	8位机器代码	适用核心	核心功能与硬件动作摘要
指针间接寻址	LDR	00110XXX	双核通用	$ACC \leftarrow RAM[(\lceil 100 \rceil + XXX) \pmod 8]$
	SDR	00111XXX	双核通用	$RAM[(\lceil 100 \rceil + XXX) \pmod 8] \leftarrow ACC$
控制流与调用	JMP	111XXXXX	双核通用	无条件跳转到 5 位目标地址 XXXXX
	BEQ	110XXXXX	双核通用	条件跳转：若 $ACC == 0$ 则跳转到 XXXXX
	CALL	101XXXXX	双核通用	压栈 PC 与 ACC，调用 XXXXX（最大 3 层）
	RET	10000000	双核通用	从调用栈弹出恢复 PC 与 ACC 并返回（低 5 位严禁乱填）
	HALT	00101000	双核通用	停机挂起当前核心（低 3 位严禁乱填）
	NOP	00000000	双核通用	空操作（DMA 延迟槽与微码占位）
硬件数据栈	PUSH	01100000	双核通用	将 ACC 压入硬件数据栈顶部 （中间位严格填 00）
	POP	01100001	双核通用	从硬件数据栈顶部弹出到 ACC （中间位严格填 00）
地址拓展机构	EXT	01110100	双核通用	翻转拓展状态，配合后续跳转实现 64 条指令跨 Bank 远跳转
存储器 DMA	DMA	01011XXB	双核通用	内存 001~100 批量传输（ $B = 0$ 读盘/收包， $B = 1$ 写盘/发包）， 后必跟 NOP
主核专属 IO	LW 110	00011110	主核独占	读取键盘就绪标志位（1 为有按键，0 为无）
	LW 111	00011111	主核独占	读取键盘 3 位键值， 并 硬件自动清零标志位

归属硬件模块	指令助记符	8位机器代码	适用核心	核心功能与硬件动作摘要
	MMIO	01111000	主核独占	激活南桥时钟，向 $RAM[110]$ 发送 $RAM[111]$ 数据（低 3 位填 000）
	LW 101	00011101	主核独占	读取由 $RAM[110]$ 选中的南桥模块输出数据
显示控制器	DISPLAY	01010JKL	主核独占	$JKL = 100$: 3 位直显; 001 $\rightarrow$ 111: 两次分步拼接 6 位输出
	CLEAR	01101010	主核独占	快速清空 5 联主显示屏全部内容（光标原地保持）
	HOME	01110010	主核独占	快速将 5 联主屏光标复位至第 0 格开头（保留显示内容）
	DISPLAY CANVAS	00101100	主核独占	向 4 联画板屏输出 $RAM[111][1:0]$ 选定帧并自动步进
单核任务切换	YIELD	01110000	双核通用	仅限 Bank 0 执行，触发微码程序 A 切换到 Bank 1（低 3 位填 000）
	FINISH	01101000	双核通用	仅限 Bank 1 执行，触发微码程序 B 切回 Bank 0（低 3 位填 000）
	CSW	01101100	微码专用	协程上下文切换（替代 CALL/RET），无栈深度消耗，双 PC 续跑
双核协同通信	WRMBX	01100100	主核独占	将主核ACC数据写入单向硬件邮箱（零 IPI 传参）
	CPUTOG	01111100	主核独占	翻转从核启停状态; 开机时强制将从核 PC 归零并从地址 00000 启动
	LW 101	00011101	从核独占	从核读取单向硬件邮箱中的参数到 ACC（零延迟直读）